UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA
CICLO 6

ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS

"PASO DE MENSAJES"

INFORME 07

SISTEMAS OPERATIVOS

PRESENTA

ALEX JEANPIER CHATA CHINO
2024-119012

21 de octubre del 2026

1. INTRODUCCIÓN

La comunicación entre procesos es un mecanismo fundamental en los sistemas operativos, ya que permite que programas independientes intercambien información y coordinen sus actividades. En GNU/Linux existen diferentes mecanismos de comunicación entre procesos (IPC), entre ellos las tuberías, los sockets, la memoria compartida y las colas de mensajes.

En esta práctica se estudia el funcionamiento de las colas de mensajes System V, un mecanismo que permite enviar y recibir mensajes mediante estructuras identificadas por un tipo. Para ello, se parte de dos programas escritos en lenguaje C que utilizan las funciones *msgget()*, *msgsnd()* y *msgrcv()*.

Posteriormente, se desarrollan dos escenarios de comunicación: una secuencia de intercambio de mensajes entre dos procesos y un esquema cliente-servidor en el que dos clientes reciben respuestas individuales mediante una única cola de mensajes. Estas actividades permiten comprender el funcionamiento de la comunicación entre procesos, el filtrado por tipo de mensaje y el comportamiento bloqueante de las operaciones de recepción.

2. OBJETIVOS

2.1. Objetivo general

Implementar y analizar mecanismos de comunicación entre procesos mediante colas de mensajes System V en GNU/Linux, utilizando el lenguaje C.

2.2. Objetivos específicos

- Comprender el funcionamiento de las colas de mensajes como mecanismo de comunicación entre procesos.
- Utilizar las funciones de la biblioteca System V IPC para crear, enviar y recibir mensajes.
- Implementar una secuencia de intercambio de mensajes entre dos procesos.
- Desarrollar un esquema de comunicación entre un servidor y dos clientes utilizando una sola cola.
- Analizar el comportamiento bloqueante y no bloqueante de las operaciones de envío y recepción.
- Verificar el funcionamiento de los programas mediante su compilación, ejecución y observación de resultados.

3. METODOLOGÍA

La práctica se desarrolló mediante la escritura, compilación y ejecución de programas en C dentro de un entorno GNU/Linux. Para la edición del código se utilizó el editor de texto nano, mientras que la compilación se realizó mediante el compilador GCC.

En primer lugar, se revisó el código base proporcionado por la guía, compuesto por un programa receptor y otro emisor. Ambos utilizan una clave para acceder a una cola de mensajes System V y una estructura que contiene el tipo y el cuerpo del mensaje.

Posteriormente, se plantearon dos implementaciones adicionales. La primera consiste en intercambiar mensajes de forma secuencial entre dos procesos, mientras que la segunda emplea un servidor y dos clientes que se comunican de manera individual mediante una sola cola, diferenciando los mensajes por su tipo.

Finalmente, se verificó el comportamiento de los programas mediante su ejecución en terminales independientes y, en el segundo escenario, mediante el comando ipcs -q, que permite consultar las colas de mensajes existentes. Las evidencias visuales correspondientes se incorporan en el análisis de resultados.

message_rec.c

```c
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MSGSZ 128

/**
Esta estructura define los elementos que tiene un mensaje
*/
typedef struct msgbuf
{
    long mtype;
    char mtext[MSGSZ]; // Un mensaje de longitud MSGSZ
} message_buf;

int main()
{
    int msqid;
    key_t key;
    message_buf rbuf, sbuf;
    size_t buf_length;

    key = 1234;
    // Este metodo trata de acceder a una cola de mensaje con llave 1234
    // y con permisos de lectura y escritura para todo el mundo
    if ((msqid = msgget(key, 0666)) < 0)
    {
        perror("msgget");
        exit(1);
    }
    // Esta es una llamada BLOQUEANTE sobre la cola de mensajes con identificador
    // msqid. El mensaje se almacenara en la variable rbuf y se leera un total de
    // MSGSZ bytes. El mensaje sera de tipo 1.
    if (msgrcv(msqid, &rbuf, MSGSZ, 1, 0) < 0)
    {
        perror("msgrcv");
        exit(1);
    }
    printf("%s\n", rbuf.mtext);
    exit(0);
}
```

message_send.c

```c
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MSGSZ 128
typedef struct msgbuf
{
    long mtype;
    char mtext[MSGSZ];
} message_buf;

int main()
{
    int msqid;
    int msgflg = IPC_CREAT | 0666;
    key_t key;
    message_buf sbuf;
    message_buf rbuf;
    size_t buf_length;

    key = 1234;

    // Invocar a la funcion msgget() con el msgflg con valor IPC_CREAT | 0666
    // provoca que se cree la cola si esta ya no existe. Si existe, se devuelve
    // en msqid el identificador de la cola con llave '1234'.
    if ((msqid = msgget(key, msgflg)) < 0)
    {
        perror("msgget()");
        exit(1);
    }
    // Esto me indica el tipo del mensaje
    sbuf.mtype = 1;
    (void)strcpy(sbuf.mtext, "Are you hearing me?");
    buf_length = strlen(sbuf.mtext) + 1;
    // En este caso el emisor o sender envia el mensaje pero no espera respuesta
    // para saber si el receptor recibio o no el mensaje
    if (msgsnd(msqid, &sbuf, buf_length, IPC_NOWAIT) < 0)
    {
        perror("msgsnd");
        exit(1);
    }
    exit(0);
}
```

Figura 1. Edición de los programas iniciales de envío y recepción de mensajes. Fuente:

elaboración propia.

```
jeanpier@jeanpiernet:~/Documents/TEMP/05_pasoMensajes/src$ ./recv
Are you hearing me?
```

```
jeanpier@jeanpiernet:~/Documents/TEMP/05_pasoMensajes/src$ ./send
```

Figura 2. Compilación de los archivos fuente en GNU/Linux. Fuente: elaboración propia.

4. ANÁLISIS DE RESULTADOS

4.1. Comunicación inicial entre emisor y receptor

En la implementación inicial, el programa emisor crea o accede a una cola de mensajes System V mediante la función *msgget()*. A continuación, establece el tipo del mensaje en 1, almacena el texto *"Are you hearing me?"* y lo envía mediante *msgsnd()*.

El programa receptor accede a la cola utilizando la misma clave y ejecuta msgrcv() para obtener un mensaje de tipo 1. Como la recepción se realiza con el parámetro de opciones igual a 0, la operación es bloqueante: si todavía no existe un mensaje coincidente, el proceso espera hasta que llegue uno o se produzca una condición de error o interrupción.

Esta implementación permite comprobar que dos procesos independientes pueden intercambiar información sin necesidad de compartir directamente sus espacios de memoria. Sin embargo, el ejemplo inicial únicamente contempla el envío de un mensaje en una dirección, por lo que es necesario ampliarlo para implementar una conversación entre procesos.

```
jeanpier@jeanpiernet:~/Documents/TEMP/05_pasoMensajes/src$ ./recv
Are you hearing me?
```

Figura 3. Recepción del mensaje enviado mediante la cola System V. Fuente: elaboración propia.

4.2. Intercambio secuencial de mensajes entre dos procesos

Para cumplir con la primera actividad, se debe modificar el ejemplo original de manera que dos procesos intercambien tres mensajes en una secuencia definida:

1. El proceso 1 envía *"Are you hearing me?"*.
2. El proceso 2 recibe el mensaje y responde *"Loud and clear"*.

3. El proceso 1 recibe la respuesta y envía *"I can hear you too"*.

La implementación requiere que cada proceso pueda enviar y recibir mensajes. Para organizar el intercambio, se pueden utilizar tipos de mensaje diferentes para cada dirección de comunicación. Por ejemplo, el proceso 1 puede enviar mensajes de tipo 1 y esperar respuestas de tipo 2; el proceso 2 realiza la operación inversa.

Este esquema permite que ambos procesos utilicen la misma cola y seleccionen los mensajes que les corresponden mediante el parámetro msgtyp de msgrcv(). De esta manera, se mantiene el orden lógico de la conversación y se evita que un proceso consuma accidentalmente un mensaje destinado al otro.

La secuencia debe ejecutarse respetando el orden de envío y recepción. Si uno de los procesos intenta recibir un mensaje antes de que el otro lo envíe, puede quedar bloqueado temporalmente. Este comportamiento no representa necesariamente un error, sino una característica de la recepción bloqueante.

message_rec_m.c

```c
int main() {
    int msqid;
    key_t key = 1234;
    message_buf sbuf, rbuf;

    // Obtener la cola de mensajes creada por el Proceso A
    if ((msqid = msgget(key, IPC_CREAT | 0666)) < 0) {
        perror("msgget (recv)");
        exit(1);
    }

    // --- MENSAJE 1: Recibir de A (mtype = 1) ---
    printf("[Process 2] Waiting Message 1 from process 1...\n");
    if (msgrcv(msqid, &rbuf, MSGSZ, 1, 0) < 0) { // Bloquea hasta recibir tipo 1
        perror("msgrcv 1");
        exit(1);
    }
    printf("[Process 2] Received from process 1: \"%s\"\n", rbuf.mtext);

    // --- MENSAJE 2: Responder a A (mtype = 2) ---
    sbuf.mtype = 2;
    snprintf(sbuf.mtext, MSGSZ, "Message 2: Load and clear.");
    printf("[Process 2] Sending Message 2 (rpt)...\n");
    if (msgsnd(msqid, &sbuf, strlen(sbuf.mtext) + 1, 0) < 0) {
        perror("msgsnd 2");
        exit(1);
    }

    // --- MENSAJE 3: Recibir de A (mtype = 1) ---
    printf("[Process 2] Waiting Message 3 from process 1...\n");
    if (msgrcv(msqid, &rbuf, MSGSZ, 1, 0) < 0) { // Bloquea hasta recibir tipo 1
        perror("msgrcv 3");
        exit(1);
    }
    printf("[Process 2] Received from process 1: \"%s\"\n", rbuf.mtext);

    printf("[Process 2] Process 2 completed.\n");

    return 0;
}
```

message_send_m.c

```c
int main() {
    int msqid;
    key_t key = 1234;
    message_buf sbuf, rbuf;

    // Crear la cola de mensajes (si no existe) con permisos de lectura y escritura
    if ((msqid = msgget(key, IPC_CREAT | 0666)) < 0) {
        perror("msgget (send)");
        exit(1);
    }

    // --- MENSAJE 1: Proceso A -> Proceso B (mtype = 1) ---
    sbuf.mtype = 1;
    snprintf(sbuf.mtext, MSGSZ, "Message 1: Are you hearing me?.");
    printf("[Process 1] Sending Message 1...\n");
    if (msgsnd(msqid, &sbuf, strlen(sbuf.mtext) + 1, 0) < 0) {
        perror("msgsnd 1");
        exit(1);
    }

    // --- MENSAJE 2: Proceso B -> Proceso A (mtype = 2) ---
    printf("[Process 1] Awaiting a response (Message 2)...\n");
    if (msgrcv(msqid, &rbuf, MSGSZ, 2, 0) < 0) { // Bloquea hasta recibir tipo 2
        perror("msgrcv 2");
        exit(1);
    }
    printf("[Process 1]:Received from process 2 \"%s\"\n", rbuf.mtext);

    // --- MENSAJE 3: Proceso A -> Proceso B (mtype = 1) ---
    sbuf.mtype = 1;
    snprintf(sbuf.mtext, MSGSZ, "Message 3: I can hear you too.");
    printf("[Process 1] Sending Messaje 3...\n");
    if (msgsnd(msqid, &sbuf, strlen(sbuf.mtext) + 1, 0) < 0) {
        perror("msgsnd 3");
        exit(1);
    }

    // Pausa breve para permitir que el Proceso B lea el ultimo mensaje
    sleep(1);

    // Eliminar la cola de mensajes de la memoria del sistema
    if (msgctl(msqid, IPC_RMID, NULL) < 0) {
        perror("msgctl IPC_RMID");
        exit(1);
    }
    printf("[Process 1] Process 1 completed.\n");

    return 0;
}
```

Figura 4. Implementación del intercambio secuencial de mensajes entre los procesos. Fuente: elaboración propia.

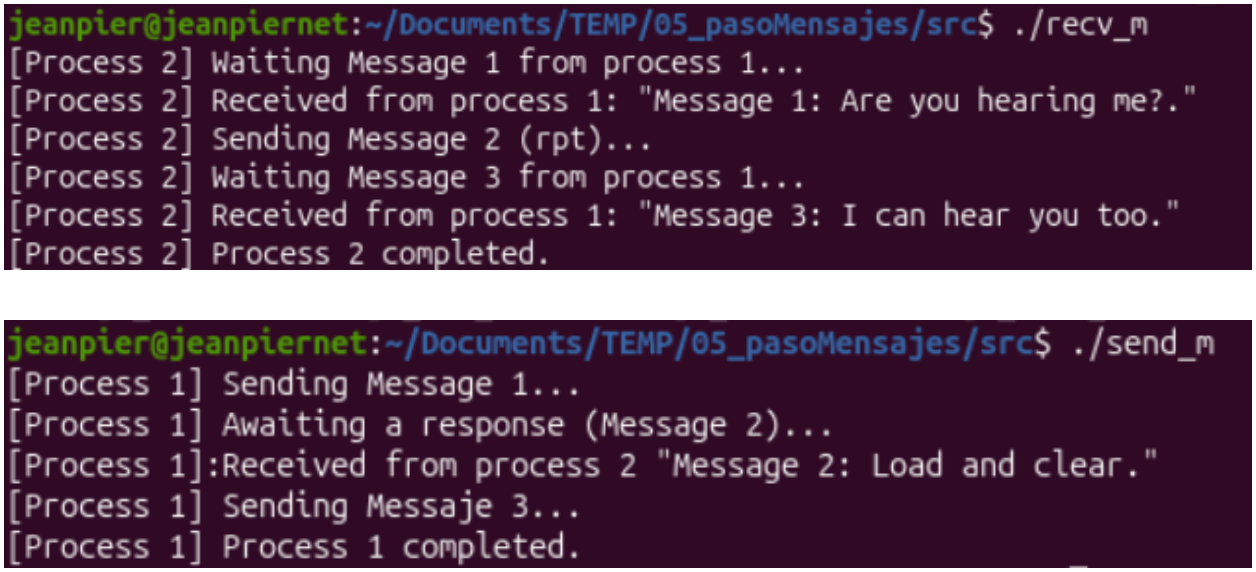


Figura 5. Evidencia de la secuencia de comunicación entre los dos procesos. Fuente: elaboración propia.

Análisis de la evidencia: Una vez realizada la ejecución, se debe comprobar que los tres mensajes aparecen en el orden previsto y que cada proceso recibe el mensaje correspondiente. Si el resultado difiere de la secuencia establecida, será necesario revisar los tipos de mensaje, la clave de la cola y el orden de las llamadas *msgsnd()* y *msgrcv()*.

4.3. Comunicación privada entre un servidor y dos clientes

La segunda actividad consiste en implementar un servidor y dos clientes independientes que intercambian mensajes mediante una sola cola System V. El objetivo es que el servidor pueda identificar cada solicitud y dirigir la respuesta al cliente correspondiente.

Para conseguirlo, se puede establecer un esquema de tipos de mensaje. Por ejemplo, el tipo 1 puede utilizarse para las solicitudes del cliente 1, el tipo 2 para las solicitudes del cliente 2 y otros tipos específicos para las respuestas dirigidas a cada cliente. La asignación exacta debe mantenerse coherente en los tres programas.

El servidor recibe las solicitudes, identifica su procedencia mediante el tipo de mensaje o la información incluida en su contenido y envía una respuesta utilizando el tipo que espera el cliente destinatario. Cada cliente ejecuta *msgrcv()* filtrando por su tipo de respuesta, de modo que no consuma la respuesta destinada al otro cliente.

server.c

```c
int main(void) {
    int qid;
    struct message msg;

    /* Crear una única cola */
    qid = msgget((key_t)QUEUE_KEY, IPC_CREAT | IPC_EXCL | 0666);
    if (qid == -1) {
        perror("Server: msgget");
        fprintf(stderr,
                "Si existe una cola anterior, elimínela con:\n"
                "ipcrm -Q 19794\n");
        return EXIT_FAILURE;
    }

    printf("Servidor iniciado. ID de cola: %d\n", qid);
    printf("Clave de la cola: 19794\n");
    printf("Esperando solicitudes de los dos clientes...\n");
    fflush(stdout);

    /* Recibir solicitud del cliente 1 */
    if (receive_message(qid, &msg, CLIENT1_REQUEST) == -1)
        goto error;

    printf("Solicitud del cliente 1: %s\n", msg.mtext);

    /* Recibir solicitud del cliente 2 */
    if (receive_message(qid, &msg, CLIENT2_REQUEST) == -1)
        goto error;

    printf("Solicitud del cliente 2: %s\n", msg.mtext);

    /* Responder individualmente mediante mtype */
    if (send_message(qid, CLIENT1_REPLY,
                     "Respuesta privada del servidor para Cliente 1") == -1)
        goto error;

    printf("Respuesta enviada al cliente 1 (tipo 1).\n");

    if (send_message(qid, CLIENT2_REPLY,
                     "Respuesta privada del servidor para Cliente 2") == -1)
        goto error;

    printf("Respuesta enviada al cliente 2 (tipo 2).\n");
    printf("Esperando confirmaciones de recepción...\n");
```

```
86
87          /* Esperar confirmación de ambos clientes */
88          if (receive_message(qid, &msg, CLIENT1_ACK) == -1)
89              goto error;
90
91          printf("Confirmación del cliente 1: %s\n", msg.mtext);
92
93          if (receive_message(qid, &msg, CLIENT2_ACK) == -1)
94              goto error;
95
96          printf("Confirmación del cliente 2: %s\n", msg.mtext);
97
98          /* Eliminar la cola cuando terminó la comunicación */
99          if (msgctl(qid, IPC_RMID, NULL) == -1) {
100             perror("Server: msgctl");
101             return EXIT_FAILURE;
102         }
103
104         printf("Cola eliminada. Servidor finalizado.\n");
105         return EXIT_SUCCESS;
106
107     error:
108         /* Limpiar la cola ante un error */
109         if (msgctl(qid, IPC_RMID, NULL) == -1)
110             perror("Server: msgctl al limpiar");
111
112         return EXIT_FAILURE;
113     }
```

client1.c

```c
int main(void) {
    int qid;
    struct message msg;

    qid = msgget((key_t)QUEUE_KEY, 0666);
    if (qid == -1) {
        perror("Client 1: msgget");
        fprintf(stderr, "Inicie primero el servidor.\n");
        return EXIT_FAILURE;
    }

    /* Enviar solicitud al servidor */
    msg.mtype = 11;
    snprintf(msg.mtext, sizeof(msg.mtext),
             "Solicitud de informacion del Cliente 1");

    if (msgsnd(qid, &msg, strlen(msg.mtext) + 1, 0) == -1) {
        perror("Client 1: msgsnd");
        return EXIT_FAILURE;
    }

    printf("Cliente 1: solicitud enviada.\n");

    /* Recibir solamente mensajes de tipo 1 */
    if (msgrcv(qid, &msg, sizeof(msg.mtext), 1, 0) == -1) {
        perror("Client 1: msgrcv");
        return EXIT_FAILURE;
    }

    printf("Cliente 1 recibe: %s\n", msg.mtext);

    /* Confirmar recepción al servidor */
    msg.mtype = 21;
    snprintf(msg.mtext, sizeof(msg.mtext),
             "Respuesta recibida correctamente");

    if (msgsnd(qid, &msg, strlen(msg.mtext) + 1, 0) == -1) {
        perror("Client 1: msgsnd ACK");
        return EXIT_FAILURE;
    }

    printf("Cliente 1: confirmacion enviada.\n");
    return EXIT_SUCCESS;
}
```

client2.c

```c
int main(void) {
    int qid;
    struct message msg;

    qid = msgget((key_t)QUEUE_KEY, 0666);
    if (qid == -1) {
        perror("Client 2: msgget");
        fprintf(stderr, "Inicie primero el servidor.\n");
        return EXIT_FAILURE;
    }

    /* Enviar solicitud al servidor */
    msg.mtype = 12;
    snprintf(msg.mtext, sizeof(msg.mtext),
             "Solicitud de informacion del Cliente 2");

    if (msgsnd(qid, &msg, strlen(msg.mtext) + 1, 0) == -1) {
        perror("Client 2: msgsnd");
        return EXIT_FAILURE;
    }

    printf("Cliente 2: solicitud enviada.\n");

    /* Recibir solamente mensajes de tipo 2 */
    if (msgrcv(qid, &msg, sizeof(msg.mtext), 2, 0) == -1) {
        perror("Client 2: msgrcv");
        return EXIT_FAILURE;
    }

    printf("Cliente 2 recibe: %s\n", msg.mtext);

    /* Confirmar recepción al servidor */
    msg.mtype = 22;
    snprintf(msg.mtext, sizeof(msg.mtext),
             "Respuesta recibida correctamente");

    if (msgsnd(qid, &msg, strlen(msg.mtext) + 1, 0) == -1) {
        perror("Client 2: msgsnd ACK");
        return EXIT_FAILURE;
    }

    printf("Cliente 2: confirmacion enviada.\n");
    return EXIT_SUCCESS;
}
```

Figura 6. Implementación del esquema cliente-servidor con una cola compartida. Fuente: elaboración propia.

```
------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x00004d52 3          jeanpier   666        78           2
```

Figura 7. Verificación de la cola de mensajes del sistema mediante ipcs -q. Fuente: elaboración propia.

```
jeanpier@jeanpiernet:~/Documents/TEMP/05_pasoMensajes/src$ ./server
Servidor iniciado. ID de cola: 2
Clave de la cola: 19794
Esperando solicitudes de los dos clientes...
Solicitud del cliente 1: Solicitud de informacion del Cliente 1
Solicitud del cliente 2: Solicitud de informacion del Cliente 2
Respuesta enviada al cliente 1 (tipo 1).
Respuesta enviada al cliente 2 (tipo 2).
Esperando confirmaciones de recepción...
Confirmación del cliente 1: Respuesta recibida correctamente
Confirmación del cliente 2: Respuesta recibida correctamente
Cola eliminada. Servidor finalizado.
```

```
jeanpier@jeanpiernet:~/Documents/TEMP/05_pasoMensajes/src$ ./client1
Cliente 1: solicitud enviada.
Cliente 1 recibe: Respuesta privada del servidor para Cliente 1
Cliente 1: confirmacion enviada.
```

```
jeanpier@jeanpiernet:~/Documents/TEMP/05_pasoMensajes/src$ ./client2
Cliente 2: solicitud enviada.
Cliente 2 recibe: Respuesta privada del servidor para Cliente 2
Cliente 2: confirmacion enviada.
```

Análisis de la evidencia: La ejecución debe demostrar que existe una única cola de mensajes y que cada cliente recibe exclusivamente la respuesta dirigida a su tipo. Si las respuestas se intercambian o un cliente queda esperando indefinidamente, se debe revisar la correspondencia entre los tipos utilizados en *msgsnd()* y los filtros configurados en *msgrcv()*.

4.4. Comportamiento bloqueante y no bloqueante

Las funciones *msgsnd()* y *msgrcv()* permiten controlar el comportamiento de los procesos durante el intercambio de mensajes.

Por defecto, *msgrcv()* se bloquea cuando no existe un mensaje que coincida con el filtro solicitado. Por su parte, *msgsnd()* puede bloquearse cuando la cola no dispone de espacio suficiente para almacenar el mensaje. Este comportamiento facilita la coordinación entre procesos, aunque también puede ocasionar esperas si los participantes no siguen el protocolo de comunicación esperado.

Al utilizar la opción *IPC_NOWAIT*, las operaciones correspondientes no esperan indefinidamente. En una recepción sin mensajes coincidentes, *msgrcv()* puede fallar con el error *ENOMSG*; en un envío que no puede completarse porque la cola está llena, *msgsnd()* puede fallar con *EAGAIN*.

Por tanto, la selección entre operaciones bloqueantes y no bloqueantes depende de las necesidades de la aplicación. En esta práctica, el comportamiento bloqueante resulta útil para coordinar el intercambio secuencial, mientras que el modo no bloqueante puede ser conveniente cuando el programa necesita continuar ejecutándose si no hay mensajes disponibles.

4.5. Funciones principales utilizadas

Función	Finalidad
msgget()	Crea una cola de mensajes o permite acceder a una existente mediante una clave.
msgsnd()	Envía un mensaje a la cola especificada.
msgrcv()	Recibe un mensaje, opcionalmente filtrado por tipo.
msgctl()	Permite consultar o modificar las propiedades de una cola y eliminarla cuando corresponda.

El uso combinado de estas funciones permite implementar una comunicación estructurada entre procesos. Asimismo, la administración de la cola es importante porque los recursos System V pueden permanecer en el kernel después de que los programas terminen. Por esta razón, conviene verificar su estado y eliminar las colas que ya no se necesiten.

4.6. Consideraciones técnicas

Durante el desarrollo de programas que utilizan colas de mensajes, es importante verificar los valores de retorno de las funciones del sistema, comprobar que los tamaños de los mensajes sean correctos y mantener una correspondencia entre los tipos enviados y los esperados por cada receptor.

También debe considerarse que la clave utilizada en el ejemplo (*1234*) es una constante fija. Aunque resulta útil para una práctica básica, una implementación más robusta puede emplear *ftok()* para generar una clave a partir de una ruta y un identificador de proyecto, controlando además posibles errores y conflictos.

Finalmente, la cola debe eliminarse de manera controlada cuando ya no sea necesaria, utilizando *msgctl()* con *IPC_RMID* o una herramienta como *ipcrm*. La eliminación debe coordinarse con los procesos que todavía la estén utilizando.

5. CONCLUSIONES

1. Las colas de mensajes System V proporcionan un mecanismo de comunicación entre procesos independientes en GNU/Linux, permitiendo intercambiar mensajes estructurados mediante las funciones *msgget()*, *msgsnd()* y *msgrcv()*.
2. El intercambio secuencial entre dos procesos demuestra la importancia de establecer un protocolo de comunicación claro, en el que los tipos de mensaje y el orden de las operaciones permitan identificar correctamente cada mensaje.
3. La comunicación entre un servidor y dos clientes puede organizarse mediante una sola cola, utilizando distintos tipos de mensaje para diferenciar las solicitudes y dirigir las respuestas. Sin embargo, este filtrado no sustituye los mecanismos de autenticación y autorización.
4. El comportamiento bloqueante de las operaciones facilita la coordinación entre procesos, mientras que *IPC_NOWAIT* permite evitar esperas indefinidas en determinadas situaciones, a costa de tener que gestionar explícitamente los errores de envío o recepción.
5. La correcta administración de las colas, la validación de los parámetros y la comprobación de los valores de retorno son aspectos esenciales para desarrollar programas IPC fiables y evitar recursos residuales en el sistema.

6. REFERENCIAS BIBLIOGRÁFICAS

Kerrisk, M. (2010). *The Linux programming interface: A Linux and UNIX system programming handbook*. No Starch Press.

The Open Group. (s. f.). *The Open Group Base Specifications: System Interfaces*. https://pubs.opengroup.org/onlinepubs/9799919799/

Linux man-pages project. (s. f.). *msgget(2) — get a System V message queue identifier*. https://man7.org/linux/man-pages/man2/msgget.2.html

Linux man-pages project. (s. f.). *msgsnd(2) — System V message queue operations*. https://man7.org/linux/man-pages/man2/msgsnd.2.html

Linux man-pages project. (s. f.). *msgctl(2) — System V message control operations*. https://man7.org/linux/man-pages/man2/msgctl.2.html